MAVRA: A Shared Memory for Multi-agent Workloads

Author Names (to be filled in)
Affiliation

***Abstract*—Data agents repeatedly explore the same database across tasks and sessions, spending LLM calls on knowledge that previous agents have already discovered. We present MAVRA, a shared memory layer between data agents and the database that reduces exploration, token consumption, and end-to-end time. The layer extracts executable metric and feature definitions from successful sessions, shares them across agents, and compiles them for new questions and prediction instances. Learning, validation, and optimization costs are amortized across consumers. On a TPC-DS workload with 16 agents, MAVRA reduces token consumption by 72% and achieves a 4.6× end-to-end speedup relative to no memory, while using 65% fewer tokens than private memory. On six RelBench retraining tasks, token use from the second cutoff onward falls by 69% on rel-stack and 50% on rel-trial relative to no memory, with comparable predictive quality. The layer also improves and maintains correctness through publication checks, snapshot-bound condition validation, and shared maintenance. TPC-DS accuracy increases from 74% to 96% initially and reaches 99% after a data restatement, compared with 55% without memory.**

***Index Terms*—data agents, shared memory, metric definitions, feature engineering, data validation**

I. Introduction

Data agents repeatedly explore the same database to answer different questions and build predictive features. An agent asking for March revenue, another asking for September revenue, and a third asking for quarterly revenue all need the same measure, grain, filters, and joins. When each session discovers these independently, the workload pays repeatedly for schema inspection, value probes, join tests, and the LLM calls that plan and interpret them. In our traces of 100 memoryless metric sessions, 93% of lookups repeat a fact obtained by an earlier session, and 71% of session time precedes the answer query. Public benchmark tasks exhibit the same overlap in calculations and domain knowledge (Section II-A).

We present MAVRA (**M**etric-**A**ware **V**alidation and **R**euse for **A**gents), a shared memory layer between data agents and the database that reduces repeated exploration, token consumption, and end-to-end time. Figure 1 contrasts independent sessions with this shared layer. Successful sessions contribute database knowledge; later agents retrieve it and obtain SQL for their own periods or prediction instances. Knowledge accumulates around the database and serves multiple agents, tasks, and sessions. A new consumer can therefore use discoveries made by previous consumers, amortizing learning over the workload.

The layer stores executable metric and feature definitions, together with table descriptions and verified join paths. A definition captures a measure, its grain, joins, and required filters;

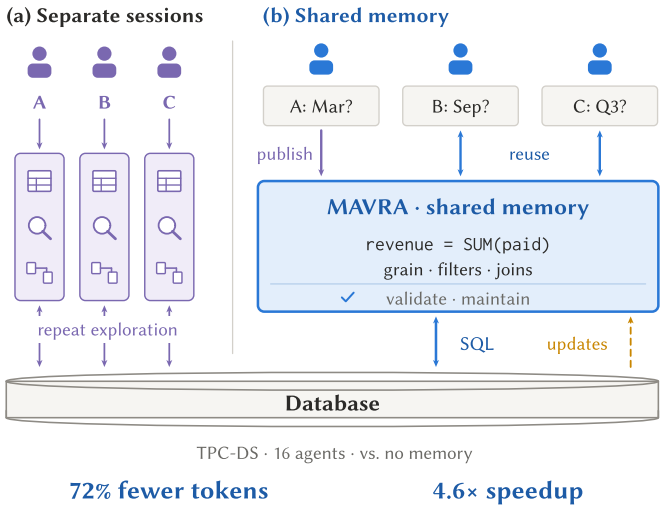


Fig. 1. Shared memory reduces repeated exploration. (a) Each agent inspects schemas, probes values, and tests joins. (b) MAVRA sits between agents and the database: successful sessions contribute definitions; later agents reuse them. Validation and maintenance are shared within the layer. Reported savings are from the 16-agent TPC-DS study (Section V-C).

feature definitions additionally encode the entity and temporal window. Compilation binds these objects to a new question or seed table. On a definition hit, compilation bypasses repeated schema, value, and join exploration, shortening the sequence of LLM and tool calls and the contexts processed by those calls. Our cost model makes the effect explicit: as more questions reuse a learned definition, the initial discovery and publication cost is spread over more uses, and per-question cost approaches the cost of reuse (Section II-B).

The shared layer also improves and maintains answer correctness. Publication checks verify extracted definitions before other agents use them. Conditions derived from grain and joins are checked on the consuming query's snapshot, and their results are shared by dependency version. Data changes trigger revalidation, checked metric repair, or revocation; feature compilation enforces per-instance temporal bounds. These mechanisms let consumers share validated knowledge and its upkeep along with the savings from reuse (Section IV).

We evaluate token and time savings on RelBench [1], TPC-DS, and TPC-H. On the 16-agent TPC-DS workload, MAVRA uses 72% fewer tokens and achieves a 4.6× end-to-end speedup over no memory; it uses 65% fewer tokens

than private memory with the same learning mechanisms. For later questions, LLM calls fall from 6.9 to 3.2 and tool calls from 10.6 to 4.4. On RelBench, token use from the second retraining cutoff onward falls by 69% on rel-stack and 50% on rel-trial relative to no memory, and by 52% on both databases relative to the most token-efficient agent-memory baseline, with comparable predictive quality across six tasks. TPC-DS accuracy rises from 74% to 96% initially and reaches 99% after a restatement, compared with 55% without memory.

The contributions are:

- A shared memory layer between agents and the database that turns session discoveries into executable metric and feature definitions, reducing repeated exploration across agents and tasks (Sections III and IV).
- A workload analysis, amortization model, and end-to-end evaluation that connect shared knowledge reuse to reductions in LLM calls, tokens, and time (Sections II and V).
- Publication validation, snapshot-bound shared checks, and maintenance mechanisms that improve answer correctness and govern reuse after data changes (Section IV).

II. Background and Motivation

A. Repeated knowledge across agent queries

Tasks over the same database require overlapping domain knowledge. We quantify this overlap using the released tasks of public text-to-SQL benchmarks. LiveSQLBench [2], on which BIRD-Interact builds [3], documents each of its PostgreSQL databases in a knowledge base of calculations, domain rules, and value meanings (1,369 entries for the 22 databases of the full release), and BIRD [4] gives each question evidence that defines its terms. Since LiveSQLBench withholds which entries a task needs, we count a task as needing an entry when its question names the entry or an entry built on it; a BIRD question needs the terms that its evidence defines. By excluding implicit references and paraphrases, these lexical counts provide a conservative estimate of knowledge repetition. Table I shows that most tasks share knowledge with another task on their database, 78% on LiveSQLBench and 51% on BIRD. Half of LiveSQLBench's uses of knowledge (50%) repeat an earlier use, and calculations, the kind of knowledge that MAVRA learns, repeat as often (53%). Repetition grows with the number of tasks: the 10th task on a LiveSQLBench database finds 48% of the knowledge it needs already needed by an earlier task, and the 20th 65%; on BIRD the share still grows to 33% by the 100th question. These benchmarks contain 27 to 139 tasks per database on average. Every task is answered by a new agent session, which explores the knowledge again.

In applications, parameterized query templates and database knowledge reside in code, so millions of queries reduce to thousands of templates [5]; a new data-agent session must acquire that knowledge through schema lookups, data probes, and checks. To see what this costs, Table II compares two workloads. The application side is Redset [6]: 441 million queries of 400 Amazon Redshift clusters over three months, analyzed per cluster with the dataset's template fingerprint [7]. To capture session-level execution and LLM overhead not logged in the static benchmarks, the agent side traces 100 memoryless DeepSeek-V4.1-Flash-1017 sessions over 25 analytical questions spanning five metrics (Section V: the learning questions and the first three scored forms). Each session uses table-listing, table-description, and SQL tools. A lookup's *fact* is its purpose together with the tables and columns it reads.

Applications repeat queries: in Redset, 93.4% of the statements repeat an earlier template, 6.6 templates per 100 statements, and prepared statements and the result cache, which answers 34% of the reads, serve them well. Agent queries have greater template diversity in our traces: 100 agent statements contain 76 templates, because each session writes its SQL

TABLE I

KNOWLEDGE THAT TASKS ON ONE DATABASE SHARE IN PUBLIC BENCHMARKS. A LIVESQLBENCH TASK NEEDS THE KNOWLEDGE-BASE ENTRIES THAT IT NAMES AND THE ENTRIES THEY BUILD ON; A BIRD QUESTION NEEDS THE TERMS THAT ITS EVIDENCE DEFINES. SHARES OF TASKS ARE OF THE TASKS THAT NEED KNOWLEDGE; THE LAST ROWS AVERAGE OVER RANDOM ARRIVAL ORDERS OF THE TASKS ON EACH DATABASE.

	LiveSQLBench		BIRD
	Full	Lite	dev
Databases	22	18	11
Tasks	600	270	1,534
needing knowledge	82%	97%	83%
sharing it with another task	78%	78%	51%
Uses of knowledge repeating an earlier use	50%	40%	28%
among calculations	53%	47%	26%
Knowledge of the n-th task already needed			
$n = 10$	48%	47%	15%
$n = 20$	65%	–	24%
$n = 100$	–	–	33%

TABLE II

APPLICATION AND AGENT WORKLOADS. APPLICATION: REDSET [6], 441 M QUERIES FROM 400 REDSHIFT CLUSTERS (READ QUERIES). AGENTS: 100 NEW SESSIONS ISSUING 521 DATABASE CALLS, 176 OF THEM SQL; A *lookup* IS A SCHEMA LOOKUP, DATA PROBE, OR VALIDATION CALL; AN LLM CALL ISSUES NO ANSWER SQL WHEN NONE OF ITS TOOL CALLS RUNS A QUERY THAT THE FINAL ANSWER USES.

	Redset	Agents
LLM calls per session	–	4.6
issuing no answer SQL	–	57%
Session time before the answer SQL	–	71%
Lookups repeating an earlier fact	–	93%
share of lookup DB time	–	77%
SQL lookups repeating a fact	–	38%
with the same SQL text	–	0%
SQL repeating an earlier template	93.4%	24%
Templates per 100 statements	6.6	76
Answered from the result cache	34%	–
SQL that fails	0.35%	1.7%
Wrong answers from SQL without error	–	32 / 32
Calls that validate	none	4.8%
Sessions checking key uniqueness	–	16%

anew from what it has just looked up. What agents repeat is knowledge: on our questions, 93% of the lookups repeat a fact that an earlier session obtained, yet no SQL probe that repeats a fact repeats the text of an earlier one, so a cache keyed by query text serves none of them. Even with exact or semantic result caching on the database side, caching probe results does not remove the LLM calls that formulate probes and interpret their outputs: 57% of an agent's LLM calls issue no SQL that the answer uses, and 71% of a session passes before that SQL starts. A cache makes a lookup cheaper for the database, but the agent still spends the LLM call that issues the lookup and reads its result. Liu et al. find the same split in the traces of an OpenAI o3-based agent: of 12.67 SQL statements per task, 3.44 explore tables and 3.56 explore columns, and expert hints such as which column holds the relevant information cut column exploration by 27.7% and all statements by 18.1% [8]. Knowledge that an agent has before it starts removes exploration; MAVRA supplies it from what other agents have learned. Incorrect answers can occur without SQL execution errors: 31 of the 32 wrong answers occur when a question names a metric without defining it, exactly where a shared definition helps. Only 16% of sessions check key uniqueness, the kind of condition that MAVRA maintains on their behalf.

B. Amortizing exploration across agents

Sharing memory across agents amortizes the discovery of database knowledge. A private memory serves the later sessions of its owner; a database-wide memory also serves other agents whose questions use the same definitions. With N agents and k metrics, private copies can require up to Nk discoveries, whereas under stable data one successful publication per metric can serve the population. MAVRA makes these publications executable for new periods and tasks, reducing both repeated discovery and the work of translating knowledge into SQL. The population study measures this effect using shared and private copies of the same learning and validation mechanisms (Section V-C).

A cost model. Let N agents each ask q questions about k metrics, let c_e be the cost of answering a question by exploration and c_r the cost of answering it from memory, and let c_ℓ be the cost of learning one definition (extraction and publication checks). Assume stable definitions and one successful publication per metric. Without memory the agents pay $Nq\,c_e$. Shared memory pays at most $k(c_e + c_\ell)$ for the first question about each metric and $(Nq - k)\,c_r$ for the rest, plus its upkeep once per data change, so its saving

$$1 - \frac{k(c_e + c_\ell) + (Nq - k)\,c_r}{Nq\,c_e} \longrightarrow 1 - \frac{c_r}{c_e} \qquad (N \to \infty)$$

grows with every agent that arrives. Per-agent memory pays the learning up to Nk times and reuses only when an agent asks about a metric it learned itself, which is rare when each agent asks a few of many questions, so its cost stays near $Nq\,c_e$. Section V-C measures both: on TPC-DS, c_e is 55.8 k tokens and c_r is 10.0 k, so the token saving approaches 82%. Writing $r = (Nq-k)/Nq$ for the share of questions answered from memory, the saving is $1 - (1-r)(c_e + c_\ell)/c_e - r\,c_r/c_e$. With $c_\ell = 5.8$ k tokens, the model gives 72% at the 89% of questions that MAVRA's agents answer from memory, the saving we measure.

Table III organizes reuse by the object being shared and the operations on it. Semantic caching reuses predicate-described data regions [14]; TxCache reuses application computations on a consistent snapshot [15]; metric layers share declared definitions and SQL generation [12], [13]. MAVRA shares definitions learned through agent sessions and compiles them for later questions. Their dependency-versioned check results, repairs, and query rewrites are shared as well, extending amortization from discovery to validation, maintenance, and execution.

III. System Overview

MAVRA places a shared memory layer between data agents and a SQL database, PostgreSQL in our prototype, to reduce repeated exploration and its token and time cost. Successful sessions contribute executable metric and feature definitions, table descriptions, and verified join paths. Later agents retrieve this knowledge and obtain SQL for their periods or seed tables. Numbered revisions, learning records, and shared check results support validation and maintenance within the layer. Figure 2 shows the publication, reuse, and maintenance paths.

Learn and publish. Agents explore through common table, join, and SQL tools. For metric questions, they submit answers with the query references and arithmetic that produced them; for prediction tasks, they submit feature SQL and validation scores. From successful sessions, MAVRA extracts structured candidates and publishes revisions only after deterministic checks (Sections IV-A and IV-E). Agents cannot write definitions directly. Table descriptions and verified join paths are also shared to reduce repeated exploration.

Retrieve and execute. For analytics, `find_metric` supplies a valid revision and its SQL. The agent passes the revision to `run_sql`, which checks the conditions on the query's snapshot before executing the SQL (Section IV-C). For prediction tasks, `find_features` retrieves definitions for the target entity. Then `compose_features` compiles the selected definitions into SQL for the seed table, with temporal bounds for each instance (Section IV-E). Missing or invalid definitions lead agents back to exploration.

Validate and maintain. After an update, MAVRA rechecks conditions whose dependencies changed, sharing matching check results across definitions and agents. It repairs affected metric definitions or invalidates them, and revokes feature definitions whose conditions fail (Sections IV-C and IV-E). Cheaper equivalent metric revisions are checked before publication and shared by all consumers, amortizing optimization as well as maintenance (Section IV-D).

IV. Design of MAVRA

Sharing saves time and tokens only if one agent's exploration answers other agents' questions, and only if keeping

TABLE III
REUSE OBJECTS AND VALIDITY MECHANISMS ACROSS DESIGNS (NR: NOT REPORTED FOR THE REUSE OBJECT).

Design	Reusable object / source	Derived conditions / shared checks	Binding to execution snapshot	Maintenance / recovery
AgentSM [9]	Structured trajectories; composite tools	NR	NR	NR for stored trajectories
G-Memory [10]	Trajectories and insights shared within a multi-agent system	NR	NR	Memory evolves with task feedback
DataLab [11]	Business logic from SQL/Python history; knowledge retrieval and DSL	NR for extracted logic	NR	Notebook dependencies updated after edits
MetricFlow / metric views [12], [13]	Centrally declared metrics; shared SQL generation	Declared joins; metric-view multiplicity may be assumed[a]	NR for data-condition checks	Definition edits shared by consumers
Semantic caching [14]	Data regions reused across different queries	Semantic query decomposition	NR	Updates outside the cited study
TxCache [15]	Cached application computations	Dependency tracking and validity intervals	Cached and database reads see one snapshot	Dependency-driven invalidation
MAVRA	Revisioned metric definitions from verified sessions	Derived from grain/joins; shared by dependency version	Conditions checked on the query snapshot	Recheck; checked repair or revocation

[a]Databricks metric views document that join multiplicity declarations (`at_most_one_match`) are assumed rather than validated at runtime [13].

the shared knowledge valid costs less than exploring again. Following the modules in Figure 2, we describe Publish and Shared memory (Section IV-A), Maintain (Sections IV-C and IV-D), and Compile and run for metric queries and predictive features (Sections IV-C and IV-E). Section IV-B establishes when the metric conditions suffice for correct aggregation.

A. What agents share and how it is learned

Shared objects. MAVRA shares database knowledge that agents discover through exploration and whose structural assumptions can be checked against data: metric definitions, including their grain, joins, filters, and encoded value meanings. It also shares table descriptions, verified join paths, and check results to avoid repeated database work. Question parameters and answers remain separate; a result cache handles exact query repeats. Agent strategies, conversations, and preferences remain in agent memory.

A definition revision is $m^r = (B, I, C, E, r)$, where B is its business meaning, I its structured implementation, C its set of executable conditions, E its learning record, and r its revision number. The implementation records the fact table, aggregate expression, grain, date join, joins, and definition filters; its joins form a many-to-one tree rooted at the fact table (a star or snowflake). Question parameters such as the month are not part of it: the generated SQL is compiled from I for any period, difference, or ranking, so one learning session serves questions that no agent has asked yet. Figure 1 illustrates how this record supports reuse and maintenance.

Extraction and publication. A candidate comes from a session with a stated business definition and an independently verified answer, supplied by a user, report, or analyst; in the benchmark evaluation, an automated oracle compares the final scalar answer with the reference answer on the same snapshot before extraction. The LLM extractor receives the question, SQL and arithmetic trace, rejected queries, and schema and join metadata, but no business rows or result values. It proposes a JSON record; the server parses its fields and derives evidence and conditions itself. Extraction is therefore model-based, whereas publication uses deterministic checks: (G1) an explicit business-definition basis; (G2) an independently judged answer and verified calculation chain; (G3) valid tables, columns, joins, expressions, and definition filters, with question parameters excluded; (G4) fact-grain uniqueness; (G5) a single read-only example query that passes execution review; (G6) unchanged learning dependencies and reproduction of the verified value by the example; and (G7) reproduction of that value by SQL compiled from the structured fields. Numeric comparisons use the learning task's declared precision. G7 tests the artifact consumers will execute, catching omissions introduced during extraction.

Use. An agent queries `find_metric` with a metric name or phrase, matched against canonical names and aliases extracted during learning. The agent receives the valid revision: its business definition, structured fields, generated SQL, and revision number. It sets its own period in the generated SQL, or wraps the SQL for a difference or a ranking, declares the revision to `run_sql`, and answers. This replaces the exploration: the agent needs no table listing, value probe, or join test. On TPC-DS, such a session takes three LLM calls for a metric whose definition needs three inner joins; agents without memory spend 8.7 calls on questions about it, because they must find the fact table, the measure, and every join themselves. Agents issue at most three memory lookups per question before falling back to standard schema exploration.

B. When shared knowledge is correct

Data-only updates break learned SQL while the schema stays the same: status-change rows, restatements, or duplicate loads violate the grain, reused dimension keys make a many-to-one join fan out, and late or reformatted date keys move facts to another period or to none [16]. A condition c is a predicate over the tables $\mathrm{dep}(c)$ with a check query Q_c that

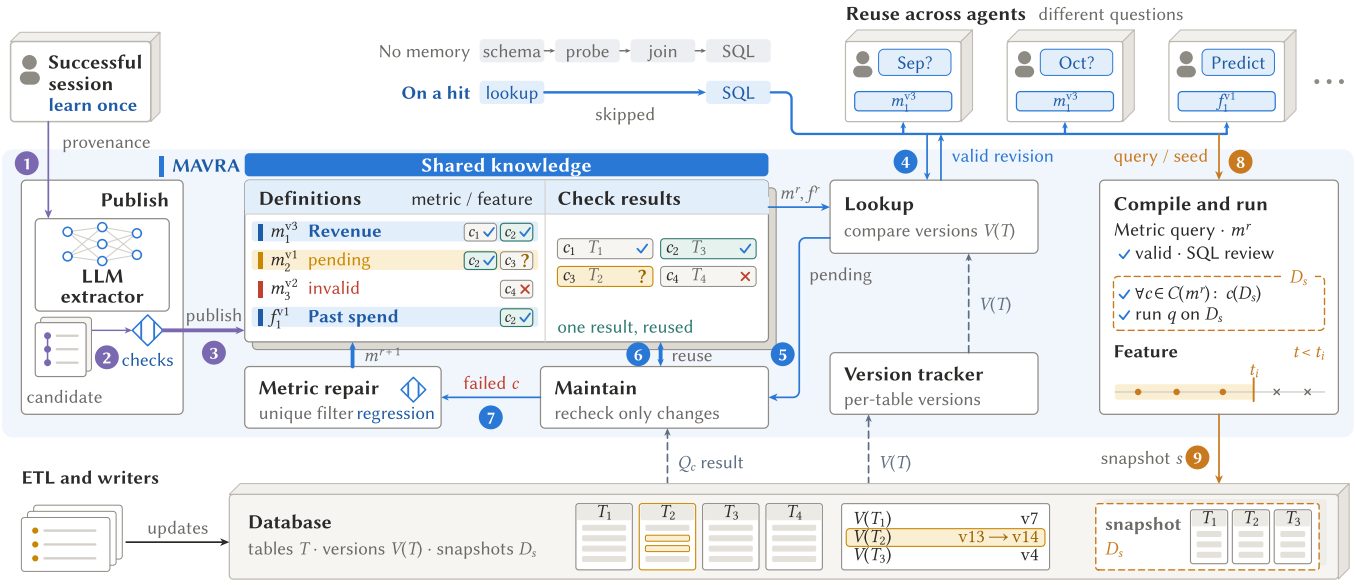


Fig. 2. MAVRA shares database knowledge to reduce repeated exploration and its token and time cost. Successful sessions contribute metric and feature definitions (1–3). On a hit (4), agents reuse them for new periods and tasks, bypassing schema inspection, value probes, and join tests. Check results, maintenance, and repairs are shared as well (5–7); execution preserves snapshot checks and per-seed temporal bounds (8–9). Thin arrows are calls, thick arrows write shared state, and dashed arrows read the database; purple, blue, and orange mark the learning, reuse, and execution paths.

returns violating rows or nothing; its normalized form $\mathrm{id}(c)$ consists of its type, tables, keys, join direction, normalized filters, and null policy. MAVRA derives four kinds from I. *Filtered key uniqueness* $\mathrm{KeyUnique}(T, K, \varphi)$: no two rows of $\sigma_\varphi(T)$ agree on K. *Join cardinality*: each row of L in a join $L \to R$ matches at most the permitted number of R rows. *Date join*: the fact's date key joins the date dimension many-to-one. *Join completeness*: for the date join and every inner join, the fraction of fact rows that match no row stays within the fraction recorded at publication plus 0.1 percentage points.

For a period p, the generated SQL evaluates $\mathrm{AGG}(e)$ over the rows of $\sigma_\varphi(F) \bowtie R_1 \bowtie \cdots \bowtie R_k$ whose date falls in p. The business definition establishes two domain premises connecting physical records to business events: (B1) the metric's business events correspond one-to-one to the values of K in $\sigma_\varphi(F)$, and (B2) e evaluates on an event's row to that event's measurement.

Proposition 1 (Sufficiency)**.** *If B1 and B2 hold on D_s and every condition in $C(m^r)$ holds on D_s, then for every period p the generated SQL of m^r returns* AGG *over the measurements of the events whose date lies in p, each event contributing exactly once; the events it omits are those whose date or join key matches no row, and the completeness conditions bound their fraction.*

Proof sketch. By B1 and filtered key uniqueness, $\sigma_\varphi(F)$ holds one row per event. Join cardinality keeps every join from duplicating a row, join completeness bounds the rows that inner joins drop, and the date join gives each row at most one date and hence at most one period. Every remaining row contributes its measurement (B2) once, to the period of its date. □

Conversely, while B1 and B2 hold, every data-only update that makes the generated SQL count an event twice or drop facts beyond the recorded fraction violates a condition.

C. Maintenance once for all agents

Shared check results. The version $V(T)$ of a table identifies the state that a check read: either the schema fingerprint, DML counters, and ETL batch identifier, or a *transactional version* that a statement-level trigger increments inside every writing transaction. A check result is keyed by $\big(\mathrm{id}(c), V(\mathrm{dep}(c))\big)$, so it serves every definition and every agent that needs the same condition on the same table versions. When an agent looks up a definition, MAVRA compares the versions recorded for each condition's tables with the current ones, keeps the check results of conditions over unchanged tables, looks up a completed result for each remaining condition at the current versions, and runs Q_c only if none exists; concurrent requests for the same check share that one execution (request coalescing). Joins are checked through shared join entries, so a changed or vanished join path also makes the definition pending. A failed condition invalidates the revision; consumers then learn that the definition is unavailable and explore on their own, and the next successful learning session publishes it again.

Update cost and scheduling. In the transactional-version implementation, a statement-level trigger performs one metadata upsert per modified table and writing statement, distributed over 16 counter shards to reduce write contention. Validation runs lazily on demand: after an update, the first consumer triggers and completes any uncached dependency checks before use; concurrent consumers coalesce onto that execution,

and subsequent consumers reuse its results while dependency versions match.

Repair once. When a fact-table grain fails, deterministic repair searches equality filters on columns with few distinct values that restore one row per business key without losing keys. Publication requires a unique passing filter to prevent ambiguous matches, such as competing primary and backup copies, and reproduction of the learning query's result through an *as-of query* on the learning snapshot. A repair runs once and serves every agent; requests that arrive while it runs join it, and a definition that cannot be repaired is invalidated, so its consumers explore until a session relearns it.

Checks at query execution. An agent declares the revisions a query uses, and `run_sql` rejects absent and invalidated ones and SQL that lacks a filter the grain requires. A passed check describes one snapshot, and a writer may commit before the query runs, so MAVRA validates on the query's own snapshot: it opens a repeatable-read transaction, looks up the check result of each declared revision's conditions at the snapshot's transactional versions, runs only the missing checks inside the transaction, and executes the query there if they hold. Looking up rather than rerunning is what keeps this cheap, and it rests on the following lemma.

Lemma 1 (Check-result reuse)**.** *For read-only snapshots under the write-coverage assumptions below, let $v_s(T)$ be the transactional version of T visible in snapshot s. If $v_{s_1}(T) = v_{s_2}(T)$ for every $T \in \mathrm{dep}(c)$, then $c(D_{s_1}) = c(D_{s_2})$.*

Proof. The visible committed-transaction sets are nested in snapshot acquisition order, say $s_1 \subseteq s_2$. A transaction that writes T increments T's counter in the same transaction, so equal versions leave no transaction in $s_2 \setminus s_1$ that writes T, and both snapshots see the same state of every table that Q_c reads. □

Write coverage. On the instrumented base tables used here (without partitioning or inheritance), the trigger covers inserts, updates, deletes (including `COPY FROM` and cascading foreign-key actions), and `TRUNCATE`; counter increments commit or roll back with the write. Reuse assumes no counter reset or reuse, so committed $A \to B \to A$ changes advance the version; operations that bypass ordinary MVCC visibility (`TRUNCATE`, table rewrites, and `COPY FREEZE`) require quiescent readers. DDL, trigger-disabled writes, replication that bypasses triggers, and restore/reset operations require cache invalidation before reuse; schema fingerprints and ETL batch markers aid detection but cannot certify unobserved writes.

D. Optimization once for all agents

A published definition is correct but not necessarily cheap, and because agents copy its generated SQL, a cheaper equivalent speeds up every consumer. After learning, MAVRA runs an optimization pass over the valid definitions; candidates are structured definitions, so compilation, the conditions, and Proposition 1 apply to them. Two rules propose candidates: *key range* replaces the period join with a range predicate on the fact table's date key, whose bounds scalar subqueries read from the date dimension, and *left-join removal* drops a left join that contributes no column and no filter. The optimizer evaluates rule candidates first. If none is adopted, an optional LLM proposal stage uses the definition, generated SQL, execution plan, and table sizes to suggest candidates. Publication is restricted to structures generated by the two rules, whose equivalence is established by Proposition 2; other proposals are retained as suggestions. A candidate must pass the static, grain, and SQL checks and three tests. *Assumption*: the key-range rule adds date-key contiguity, checked before adoption and maintained afterwards. *Equivalence*: candidate and current revision return equal results for the learning question as of learning time and, within one repeatable-read snapshot, for the twelve months of the learning year, a quarter, a period difference, and a monthly ranking. *Cost*: in the same snapshot, the paired execution times of at least eight periods, measured in alternating order, must have a 95% confidence interval below zero and a mean saving of at least 10%.

Proposition 2 (Rule equivalence)**.** *Let the date dimension's key be unique on D. (a) If* date-key contiguity *holds on D, i.e., the keys of every month form an integer interval that begins one above the previous month's last key, then for every period of consecutive months the generated SQL of a key-range revision returns on D what that of its previous revision returns. (b) If the right side of a removed left join is unique on the join key on D, removing the join does not change the result.*

Proof sketch. (a) By contiguity, the keys of a period's months are exactly the integer interval between the bounds, so a fact row's key lies in the interval if and only if it joins one date row of the period. (b) A left join with at most one match keeps each left row once and supplies nothing that is read. □

The assumption of (a) is a condition of the new revision; that of (b) is a condition of the previous revision that the new revision no longer needs. Wherever the maintained conditions hold, a rule revision thus agrees with its previous revision on every period, and the sampled tests check the implementation and measure the cost. Maintenance checks an added assumption last; when it alone fails, every condition shared with the previous revision has just been confirmed, and MAVRA invalidates the optimized revision and makes its previous revision current again.

E. Prediction tasks: shared feature definitions

Many tasks of data agents end in a model rather than in one answer: given seed rows of an entity and a seed time, the agent writes SQL that returns, for every seed, aggregates of that entity's data before the seed time, and a fixed learner is trained on them. The forecasting tasks of RelBench have this form [1], and an agent spends most of such a session exploring tables and probing values before its first features are

validated. MAVRA shares the same kind of object for these tasks. A *feature definition* is a metric definition (fact, measure, grain, verified many-to-one joins, required filters) with three more fields: the entity it describes, reached from the fact through a foreign key or a verified join (optionally through an anchor column, such as the owner of a seed post); a look-back window; and the value that a seed without rows takes, as the submitted SQL filled it in. MAVRA compiles a definition into one lateral aggregate per seed whose predicate bounds every row by the seed's time and the window, so a compiled feature cannot read data recorded after the seed time. A *task program* is the set of definitions that a successful session submitted for a task, with its validation score and the complete SQL of the submission. The complete SQL can also express features such as self-joins; consumer sessions validate its predictive quality and audit temporal leakage, while structured definitions additionally receive the checks below.

MAVRA learns these objects as it learns metrics. After a session whose features beat the entity-table reference, an extractor turns the submitted SQL into definition drafts, and a draft is published only if four checks pass on the current snapshot: its fields are valid (columns exist, expressions read only the tables involved, every join is many-to-one); the fact's grain key is unique; the compiled SQL reproduces the submitted column value by value on 2,000 seeds; and recomputing it on views that hide every row after the seed time changes no value. An agent starts with `find_program`, which returns its task's latest program, both the complete SQL, after checking that it still runs on the current snapshot, and the SQL compiled from the program's still-valid definitions; `find_features` returns the definitions published on the task's entity by any agent, compiled for the agent's seed table, and `compose_features` compiles a chosen set. When new data arrive, the conditions of every definition are checked again as in Section IV-C, and a definition whose condition fails is revoked: unlike a metric, a feature has no single reference value against which a repair could be checked.

V. Evaluation

We measure how shared memory reduces exploration, token consumption, and end-to-end time, and how these savings grow across agents and sessions. RelBench evaluates reuse across prediction tasks and retraining cutoffs; metric workloads measure sharing across consumers and the costs of learning, checking, and execution. We then evaluate answer correctness after data changes and snapshot validation under concurrent writes.

A. Published memories on a public benchmark

Benchmark and timeline. We use two databases of RelBench [1]: rel-stack (seven tables of a Stack Exchange site; tasks user-engagement, user-badge, post-votes) and rel-trial (fifteen tables of clinical trials; study-outcome, study-adverse, site-success), with RelBench's own task definitions and metrics. We evaluate periodic retraining: five cutoffs per database, the last one RelBench's official split, each earlier one moved back by the task's window, with labels regenerated by RelBench's code (those of the last cutoff equal the official tables row by row). Between cutoffs, the rows of the next period are appended to the database. Each task has its own agent, which starts a new session at every cutoff, so a memory can help an agent across cutoffs (its own task, newer data) and across tasks (other agents on the same database).

Sessions. The no-memory and four memory arms use the same agent: DeepSeek-V4.1-Flash-1017 through DeepSeek's API in thinking mode with low effort, the same model for memory operations under their documented inference settings, and the same tools (schema, joins, read-only SQL, `validate_features`, `submit`). Feature SQL returns one row per seed, may use only rows dated at or before the seed time, and runs for at most 600 s; `validate_features` trains a fixed LightGBM [17] on a 50,000-seed training sample and returns the validation score. A session ends after 40 LLM calls, 8 validations, or 2 validations in a row without improvement. From the second cutoff on, the previous cutoff's model is refit on the new data to set a validation-score target. Once the agent reaches that target, it has one further improvement attempt within 5 LLM calls before the session ends. Every validation is audited for leakage on 5,000 seeds by recomputing the features on views that hide later rows; leaking validations do not count, and leaking submissions are refused. Test labels are never seen by an agent or a memory. A session that ends without a valid solution is scored as the entity-table reference.

Baselines. We compare with five published methods, using official prompts and code where available. Three are agent memories: ACE [18], one playbook per agent updated after each session by its official reflector and curator without ground truth; G-Memory [10], its official code, one memory shared by all agents of a database; and AWM [19], one workflow memory per agent, induced again after every session from its successful sessions with the official prompt. Two are data-preparation systems of the database community, which search for each task anew: DeepPrep [20], an LLM agent whose official code explores operator pipelines as a tree with execution feedback (its reasoning loop, prompts, operators, and executor run unchanged on the task's tables with the study's model; it explores on 2,000 seeds, and its solution then runs on all seeds within the same 600 s per split), and FeatPilot [21], which searches join paths for the features that most improve the model without an LLM (our implementation follows the paper, with every aggregate bounded by the seed time). A method that needs a success signal uses the same one, a validation score better than the entity-table reference. The no-memory agent is the same agent without any memory. Implementation configurations and reproduction instructions are provided in the artifact.

The workload. Without memory, a session takes a median of 20 LLM calls and 439 k tokens, 85% of its tool calls inspect the schema or probe values, and 99% of the schema calls in later sessions repeat a call that an earlier session on the same database already made.

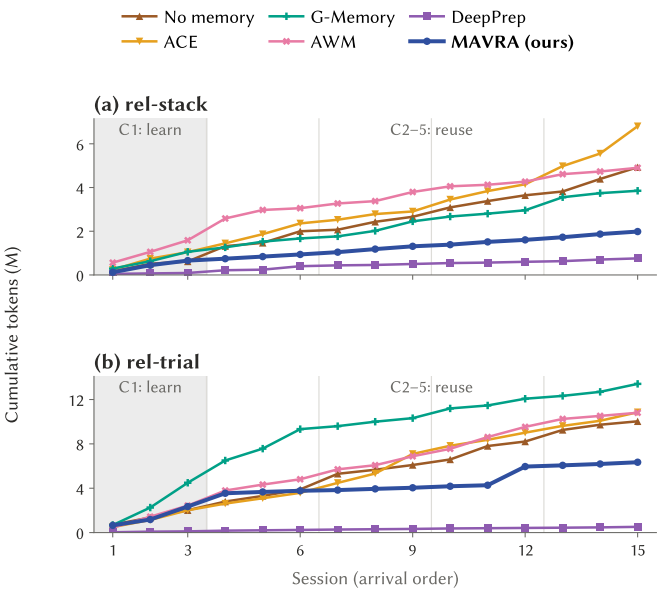


Fig. 3. Cumulative tokens, including memory operations, for the six LLM-based methods. Panels share the session axis; shading marks cutoff 1 (learning), followed by cutoffs 2–5 (reuse). Thin rules separate cutoffs.

Cost accounting. USD costs are recomputed from logged usage at the official off-peak rates recorded on October 22, 2026: per million tokens, $0.003 for cached input, $0.15 for uncached input, and $0.60 for output.

Results. Table IV reports reuse costs, per-task test scores, valid-session counts, and tasks retaining baseline predictive quality; Figure 3 shows how token costs accumulate. From the second cutoff on, MAVRA uses 69% fewer tokens than agents without memory on rel-stack and 50% fewer on rel-trial. The smaller USD reduction reflects token mix: on rel-stack, cached input falls from 86% to 67% of tokens and output rises from 11% to 15%; most avoided tokens are inexpensive cached input. Against the agent memory that uses the fewest tokens, MAVRA uses 52% fewer tokens on rel-stack and 52% fewer on rel-trial. Counting the first cutoff too, where its agents explore like the others and also pay for learning, MAVRA still uses 60% and 37% fewer tokens than no memory over all sessions. MAVRA preserves predictive quality across all six tasks: 3/3 on rel-stack and 3/3 on rel-trial meet the evaluation criteria, $\mathrm{AUC} \geq \mathrm{AUC}_0 - 0.01$ or $\mathrm{MAE} \leq 1.02\,\mathrm{MAE}_0$, where subscript 0 denotes the no-memory baseline.

Session outcomes and measurement. DeepPrep produces no valid solution in 7/15 sessions on rel-stack and 6/15 on rel-trial; these receive the reference score. Eleven failures occur when executing the final pipeline on the full seed split exceeds 600 s (five on rel-stack, six on rel-trial); two rel-stack searches terminate after exceeding the official five-error limit. FeatPilot completes all sessions without LLM calls. The evaluation comprises 210 sessions: seven methods, two databases, and one five-cutoff sequence with three tasks per cutoff. Token volume and LLM/tool call counts are the primary efficiency measures. Table IV also reports end-to-end wall-clock durations (which include external LLM API latency); full execution and measurement details are documented in the artifact.

Where the savings come from. A MAVRA agent retrieves its task's validated program alongside compiled entity-level feature definitions available from prior sessions on the same database. The complete program SQL provides an executable starting point, while the definitions support feature selection and refinement. Reuse typically reaches the target at the first validation, leaving the remaining budget for targeted improvement. These results evaluate the combined reuse path. Per session from the second cutoff on, MAVRA's agents make 7.2 schema and probe calls on rel-stack, while agents without memory make 22.1. The other memories hand the agent text about earlier work, namely a playbook (ACE: 22.6), workflows (AWM: 23.0), or trajectories and insights (G-Memory: 11.4), which the agent still turns into SQL over tables it inspects again; DeepPrep and FeatPilot keep nothing and search every task and cutoff from the start.

B. Metric questions: questions and setup

The metric studies ask how savings grow with agent count (Q1), where per-question savings arise and what learning costs (Q2), what shared maintenance and optimization cost relative to per-agent work (Q3), and whether savings survive data changes (Q4). Agents answer with one SQL query. The machine has 48 cores, 98 GiB RAM, and PostgreSQL 18.6. The population study uses TPC-DS SF1 (24 tables, 2.9 million store-sales rows); per-question and data-change studies use synthetic retail data with TPC-DS-style columns (one million store-sales rows, about 396,000 returns and 500,000 catalog sales) and TPC-H SF1.

Workload. Five metrics (store revenue, return amount, return rate, catalog revenue, electronics store revenue) each have two learning questions stating the business definition and five scored questions naming only the metric: a new period, a period difference, the highest month, the largest monthly rise, and months above the monthly mean. TPC-DS uses the same seven forms for five template-derived metrics: store sales for one category manager, registered-customer sales, mid-sized-store coupons, returns at known stores, and yearly totals. Reference queries, handwritten or compiled from TPC-DS/TPC-H templates, compute answers on the question's snapshot. Agents and extractor use DeepSeek-V4.1-Flash-1017 with high reasoning effort, 20 turns, and a 60 s SQL timeout (120 s on TPC-DS/TPC-H); methods share tools, with lookup added for memory. Low effort bounds RelBench's repeated feature-search cost; high effort prioritizes exact metric answers, with agent effort fixed across compared arms within each study. In the population study (Section V-C), table descriptions, verified joins, check results, and query results follow the memory scope: per-question without memory, per-agent with private memory, and global with MAVRA. In Sections V-D and V-F, memoryless baselines also share these resources, isolating executable definitions and maintenance. Table V lists

TABLE IV
RELBENCH: PREDICTIVE QUALITY, VALID SOLUTIONS, AND REUSE COST. EACH METHOD RUNS THREE TASKS AT FIVE CUTOFFS PER DATABASE.

rel-stack

	Reuse cost: cutoffs 2–5						Mean test score: cutoffs 1–5			Quality summary	
Method	Tokens (M)	Token ratio	LLM calls	Tool calls	Obs. min	Cost (USD)	post-votes MAE ↓	user-badge AUC ↑	user-engage. AUC ↑	Valid sessions	Quality tasks
No memory	4.31	1.00	201	334	112	0.31	0.0645	0.8588	0.8843	15/15	3/3
ACE [18]	5.76	1.34	232	339	185	0.44	**0.0637**	0.8554	0.8193	14/15	2/3
G-Memory [10]	2.79	0.65	95	181	83	0.25	0.0643	**0.8601**	**0.8852**	15/15	3/3
AWM [19]	3.32	0.77	156	324	112	0.27	0.0642	0.8600	0.8828	15/15	3/3
DeepPrep [20]	0.67	0.15	27	42	151	0.15	0.0670	0.7180	0.7604	8/15	0/3
FeatPilot [21]	–	–	–	208	59	–	0.0654	0.8296	0.8774	15/15	2/3
MAVRA (ours)	1.33	0.31	76	128	37	0.16	0.0648	0.8575	0.8834	15/15	3/3

rel-trial

	Reuse cost: cutoffs 2–5						Mean test score: cutoffs 1–5			Quality summary	
Method	Tokens (M)	Token ratio	LLM calls	Tool calls	Obs. min	Cost (USD)	site-success MAE ↓	study-adverse MAE ↓	study-outcome AUC ↑	Valid sessions	Quality tasks
No memory	8.02	1.00	319	501	138	0.32	0.4245	40.85	0.6732	14/15	3/3
ACE [18]	8.84	1.10	296	444	134	0.54	0.4188	39.64	**0.6753**	15/15	3/3
G-Memory [10]	8.92	1.11	180	290	89	0.55	**0.4074**	**39.60**	0.6658	15/15	3/3
AWM [19]	8.37	1.04	314	471	72	0.38	0.4276	39.99	0.6664	14/15	3/3
DeepPrep [20]	0.41	0.05	15	26	466	0.09	0.4516	46.04	0.5899	9/15	0/3
FeatPilot [21]	–	–	–	236	72	–	0.4267	44.16	0.6388	15/15	1/3
MAVRA (ours)	4.00	0.50	144	222	62	0.21	0.4154	41.01	0.6685	15/15	3/3

Cost columns total 12 sessions at cutoffs 2–5, including memory operations; token ratios are relative to no memory. Dashes: not applicable (FeatPilot uses no LLM). Test scores average all five cutoffs, using the entity-table reference for sessions without a valid solution. *Valid sessions*: non-fallback solutions (/15). *Quality tasks*: tasks retaining no-memory predictive quality (/3), with $\mathrm{AUC} \geq \mathrm{AUC}_0 - 0.01$ or $\mathrm{MAE} \leq 1.02\,\mathrm{MAE}_0$. Subscript 0 denotes no memory; best task means are bold. Time and USD accounting are specified in Section V-A.

TABLE V
COMPARED METHODS, BY WHAT LATER SESSIONS GET. P: POPULATION STUDY (§V-C); Q: COST PER QUESTION (§V-D); C: COST OF SHARING (§V-E); D: DATA CHANGES (§V-F).

Method	What later sessions get	P	Q	C	D
No memory	nothing; every question is explored	•	•		•
Single-agent memory	MAVRA's memory, but each agent uses only what it learned itself	•		•	
Example retrieval	up to three similar past sessions (question, SQL, calculation), never maintained				•
+ self-verification	the same, told to verify grain, joins, and date keys			•	
Invalidate on every write	definitions, dropped on any write and relearned			•	
Full recheck per definition	definitions, all conditions rechecked on use, no shared results			•	
MAVRA (ours)	one memory for all agents; definitions checked, maintained, and optimized once	•	•	•	•

TABLE VI
WHAT ONE QUESTION COSTS ONCE THE METRICS ARE LEARNED: THE SECOND HALF OF THE 16 AGENTS ON TPC-DS, MEANS OVER 5 RUNS. LLM: LLM CALLS OF THE AGENT; TOOLS: TOOL CALLS; TOKENS INCLUDE EXTRACTION.

	LLM	Tools	Tokens	Time	Correct
No memory	6.9	10.6	55.8 k	47.2 s	72%
Single-agent memory	6.2	10.5	48.3 k	40.8 s	74%
MAVRA (ours)	**3.2**	**4.4**	**10.0 k**	**6.6 s**	**98%**
MAVRA vs. no memory			5.6×	7.1×	

ablations of memory scope, example retrieval, and condition maintenance. Tokens count input plus output; agent time spans the first call to the final answer.

C. Sharing across a population of agents

Protocol (Q1). 16 agents arrive sequentially at the TPC-DS database. Each draws seven questions from the 35-question workload in Section V-B; all methods receive the same draw within a run. Memory starts empty. A correctly answered learning question triggers extraction and publication unless it already reused a definition. Memoryless agents retain no state across questions. The single-agent baseline uses the same learning, validation, and maintenance mechanisms as MAVRA, but restricts each agent to its own definitions, table descriptions, and join paths. Four methods—no memory, single-agent memory, example retrieval, and MAVRA—each run 5 times: $4 \times 16 \times 7 \times 5 = 2{,}240$ initial sessions and $4 \times 16 \times 4 \times 5 = 1{,}280$ post-restatement sessions (Section V-F), totaling 3,520. Initial-stage results below and post-restatement results use their respective denominators. Session tokens include any extraction they trigger.

What sharing saves. Figure 4 shows cumulative cost. Over 16 agents, MAVRA uses 1.81 M tokens, 72% fewer than no memory (6.43 M; savings 64–80% per run), finishes 4.6×

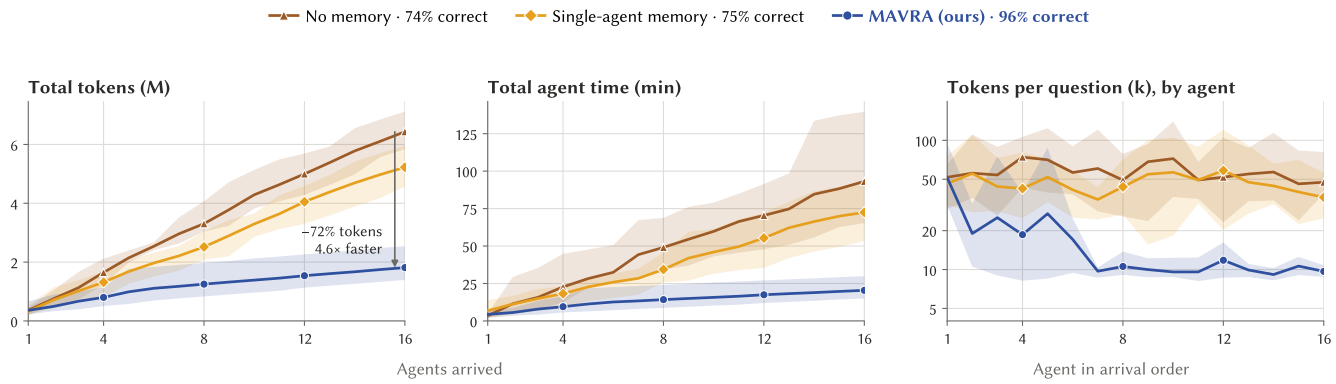


Fig. 4. What 16 agents arriving at one TPC-DS database pay, by memory design. Left and middle: total tokens (including learning) and total agent time of the first N agents. Right: tokens per question of each agent in arrival order (log scale): with MAVRA the first agents pay for learning and every later agent reuses. Lines are means over 5 runs, bands span the runs; the legend gives each method's accuracy.

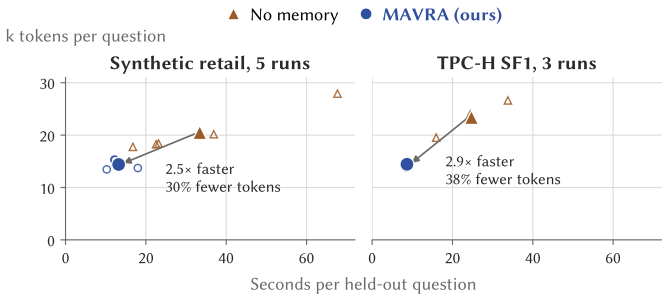


Fig. 5. What a consumer pays per held-out question on unchanged data: agent-session time and tokens per question. Small hollow markers are single runs, large markers pool all questions of a method; the arrow runs from exploring alone to MAVRA.

faster, and answers 96% correctly versus 74%. Token savings and speedups grow from 51% and 2.4× after four agents to 62% and 3.4× after eight. As Section II-B predicts, token savings approach $1 - c_r/c_e = 82\%$ as reuse grows. Early sessions perform 6 extractions per run for five metrics; 89% of all questions reuse a definition. Table VI shows why: reuse replaces schema, value, and join probes, reducing LLM calls from 6.9 to 3.2 and tool calls from 10.6 to 4.4 per question. Avoiding calls with growing contexts cuts tokens 5.6× and time 7.1×; database time falls from 1.90 to 0.24 s. In a separate 216-session study, LLM calls account for 98.9–99.6% of agent time when using shared memory.

Why memory must be shared. Private memory extracts 33 times per run but reuses on only 19% of questions: individual agents seldom revisit learned metrics. It spends 5.22 M tokens over 16 agents, including 48.3 k per late question; MAVRA uses 65% fewer tokens and runs 3.5× faster.

D. Cost per question

Reusing agents (Q2). On the synthetic retail dataset and on TPC-H, MAVRA's learning agent builds a library from ten learning questions (two per metric), and new user agents then answer held-out questions on unchanged data that give only the metric name; each method runs five times on the synthetic dataset and 3 times on TPC-H, every run learning its own library. A reusing agent finds the definition, runs its generated SQL for the question's parameters, and answers, instead of exploring tables, values, and joins. Figure 5 shows what this saves: on the synthetic dataset a question takes 13.2 s and 14.4 k tokens with MAVRA against 33.4 s and 20.5 k exploring alone, 2.5× faster (1.3–5.5× per run) with 30% fewer tokens, and on TPC-H 2.9× faster (1.8–4.0×) with 38% fewer tokens. Because reusing agents are also right more often (100% against 58%), a correct answer costs 60% fewer tokens (14.4 k against 35.6 k).

Building the memory. One library costs 193 k tokens and 420 s for the ten learning sessions, extraction, and the publication checks, paid once for all agents. The checks are what make the shared result safe to hand out: over 664 sessions with a verified answer, the replay of the generated SQL rejected 8 candidates into which extraction had introduced an error, and the calculation check 5 more. In the worst case the extractor dropped the category filter of electronics store revenue: the producing agent had answered 2,740,677.00, and the SQL every later agent would have received returns 13,521,119.40, 4.9 times as much.

E. The cost of sharing, paid once

Maintenance once, not per agent (Q3). After the population study's restatement, MAVRA repairs each affected definition once, using 258 s of database time per run; private memory repairs per-agent copies in 1,471 s. End-to-end maintenance costs 5.9 s per non-breaking update and 40 s per repaired change. Per-agent verification of shared SQL's grain, joins, and date keys adds 3 s and 2.3 k tokens per question even on unchanged data; discarding knowledge on every write costs 1.59 M relearning tokens per run.

Shared check results. Static analysis of all 99 TPC-DS templates yields 175 definitions from 82 templates, grouping each SELECT block's aggregates by fact table. Updating `store_sales` touches 101 definitions but only 12 distinct

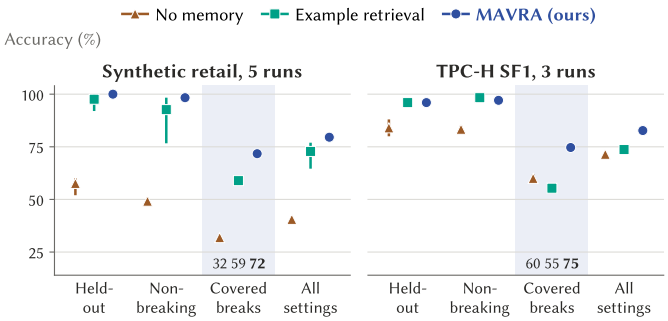


Fig. 6. End-to-end accuracy on two schemas, DeepSeek-V4.1-Flash-1017. Groups pool the held-out questions, the four non-breaking updates, the five breaking changes the conditions cover (shaded, pooled values printed), and all twelve settings, each setting weighted equally; a line spans the runs of a method and the marker gives its pooled value.

conditions. To measure sharing, a controlled benchmark publishes 4–19 definitions in four families; eight scripted agents use all definitions after each of four updates. Full rechecks cost 63.6–227.0 s as definitions increase, while MAVRA stays at 27.0–32.5 s, or 14.0% of full-recheck time at 19 definitions. Rechecking only changed tables without reuse costs 83–94%. Without shared conditions, all methods cost 95–102% of full rechecking, isolating sharing's contribution. Recheck-time ratios remain low at 4 million rows (13.7%), with 32 agents (13.6%), and up to 16 million rows (12.0–13.0%).

One optimization speeds up every agent. We evaluate an optimization pass over the 9 learned definitions on the one-million-row retail workload. 9 of the 9 published definitions received a key-range revision that reduced query execution time by 15.6–36.1% relative to the previous revision (mean 24.0%, every 95% interval below zero), with equal results on 14 periods each, and 9 of the 15 held-out queries ran the faster form because agents copy the SQL they retrieve. A separate system-level test publishes and optimizes 4 definitions (store revenue, return amount, return rate, and electronics revenue), then re-keys one month of the date dimension so that only date-key contiguity fails: maintenance invalidated all 4 key-range revisions in this test and restored their previous revisions within 8,037 ms each, and all 20 answers served afterwards are correct, whereas the key-range SQL would have been wrong by up to 324%.

F. Savings under data changes

After a restatement (Q4). Following the population study, a restatement corrects store sales while retaining old amounts as non-current rows. Each of the 16 agents returns with four affected questions. With each definition repaired once, MAVRA answers 99% correctly at 11.2 k tokens per question; agents without memory answer 55% at 84.8 k. Example retrieval remains cheap (8.2 k) but answers 59% correctly: its unmaintained SQL double-counts sales, and every wrong answer is 8–15% too high.

TABLE VII
FIXED-LIBRARY REPLAY ON TPC-DS, TPC-H, SSB, AND BIRD FINANCIAL: 1,641 COMMON QUESTION INSTANCES, MAVRA WITH THE LEARNING-SNAPSHOT REFERENCE. REFUSALS ARE UNAVAILABLE DEFINITIONS, NOT CORRECT ANSWERS; PARENTHESES COUNT REFUSALS WHERE THE ORIGINAL SQL WOULD STILL BE CORRECT.

Change group	Correct	Wrong	Refused
Non-breaking (4)	534	0	0 (0)
Targeted structural (5)	432	3	405 (51)
Unit / backup copy (2)	89	115	63 (21)

Eleven changes on two schemas. The end-to-end study repeats affected questions after four non-breaking updates, five targeted structural changes (status rows, restatements, duplicate loads, SCD Type 2, and date-key changes), and two changes outside the conditions (Figure 6). On the targeted changes, MAVRA and example retrieval answer 72% and 59% correctly on the synthetic data, and 75% and 55% on TPC-H.

Preventing invalid reuse. Table VII separates service from refusal on fixed-library replays over four benchmark databases. Across the five targeted structural scenarios, maintenance blocks 354 uses whose original SQL would be incorrect, with 3 incorrect results served. A further 51 uses are conservatively withheld because their conditions fail even though the original SQL still yields the reference answer. Revocation directs the agent to standard exploration. The unit/backup group marks the boundary between structural invariants and value semantics: 115/267 instances return incorrect results (43% overall; 56% of the 204 served answers), and 63 are refused. The two scenarios expose different requirements: unit scaling changes measure values while preserving grain and join cardinalities; an appended backup can violate grain yet leave multiple filters that each restore uniqueness. Future work will add unit-aware value checks and explicit source-of-record constraints to address these respective gaps.

All three structural-scenario errors come from one TPC-H Q12 definition. Its period uses `l_receiptdate`, but its filter compares `l_shipdate < l_commitdate`. The date-key change appends rows with null `l_shipdate`; receipt-date coverage still passes, while SQL's null comparison drops those rows. The September total is 4,149 instead of 6,107, the period difference is -9 instead of 1,949, and the highest month is 3 instead of 10. Extending completeness checks to the auxiliary filter columns `l_shipdate` and `l_commitdate`, alongside the primary date key `l_receiptdate`, would detect this failure. In the concurrent grain-breaking stress test, same-snapshot validation prevents unsafe execution; precheck-then-execute admits up to 4.6% unsafe queries.

VI. Related Work

a) Agent memory: Agent memories retain facts and conversations [22], [23], workflows [19], strategies [24], [25], playbooks [18], linked notes [26], and plans [27]. G-Memory shares hierarchical trajectories and insights within a multi-agent system [10]; Learning to Share reuses intermediate steps within a task [28], while Collaborative Memory and MemClaw

govern access and writes across users or agents [29], [30]. Recent systems also support invalidation, version checks, transactions, and repair [31]–[34]. MAVRA uses the database as a sharing boundary: agents contribute executable metric and feature definitions that later sessions compile for their own questions. Shared validation and maintenance support this reuse across the workload.

b) Database exploration memory: AgentSM structures prior or synthesized trajectories, retrieves similar questions within a database, and constructs composite tools from recurring tool sequences [9]. These mechanisms reduce repeated exploration. MAVRA represents reusable work as metric and feature definitions shared by the agents of a database. Compilation binds their measures, grain, joins, and filters to new periods and seed tables, replacing repeated exploration and SQL construction. Shared checks and maintenance support continued use of these definitions.

c) Data agents and business semantics: Agent-first data systems motivate persistent database knowledge [8], [35]; enterprise text-to-SQL and BI systems benefit from explicit business semantics [3], [4], [36]–[39]. DataLab extracts metadata and calculation logic from historical SQL/Python scripts, organizes knowledge for retrieval, and translates requests through a DSL for downstream generation [11]. It also updates notebook dependency graphs after edits. MAVRA accumulates executable definitions in a shared layer between agents and the database. Definitions learned in one session serve subsequent agents and tasks, amortizing discovery and SQL construction; validation and maintenance are performed through the same layer.

MetricFlow and Databricks metric views centralize metric definitions and query generation [12], [13]. MAVRA populates its shared layer from successful agent sessions, allowing knowledge to accumulate with database use. Compilation reuses definitions across periods and prediction instances; condition checks and maintenance keep revisions available to subsequent consumers as the database changes.

Prediction over relational data. RelBench poses forecasting tasks over multi-table databases [1] and is the testbed of relational deep learning [40]; LLM agents engineer features for such tasks by writing and validating code or SQL [41], [42]. The database community automates data preparation for learning: FeatPilot searches join paths for the features that most improve a model [21], CtxPipe builds preparation pipelines for one table by reinforcement learning [43], DeepPrep explores operator pipelines with an LLM agent over a search tree [20], and KnowTrans transfers knowledge between data-preparation tasks by fine-tuning [44]. These works prepare data for one task at a time; MAVRA makes the features that one agent verified reusable by every later session and every other task on the same database, and compiles them so that they cannot read data after the seed time.

d) Sharing work in database systems: Databases have long avoided repeated work: multi-query optimization and shared execution evaluate the common work of concurrent queries once [45]–[47], recycling keeps intermediate results and invalidates those affected by updates [48], and caches reuse results through semantic descriptions [14], update-driven invalidation [49], input-dependent keys [50], leases that merge concurrent misses [51], and transactional consistency [15]. Materialized views are maintained incrementally [52], [53], deferred off the update path [54], or refreshed when referenced [55], and build systems verify dependency traces [56], [57]. MAVRA extends work sharing to agent exploration: learned definitions replace repeated discovery and SQL construction in later sessions. Dependency tracking and snapshot consistency also allow consumers to share validation evidence for those definitions (Lemma 1).

e) Constraints and data validation: Summarizability research identifies disjointness and completeness as conditions for correct aggregation [58]–[60], integrity checking specializes verification to the updates that can violate a constraint [61], [62], and data-validation systems and pipeline tests check declared constraints on tables [63]–[65]. Building on foundational summarizability and integrity-checking primitives, MAVRA automatically derives these conditions from learned definitions and integrates their shared verification into query admission and repair.

VII. Discussion and Conclusion

MAVRA reduces repeated exploration by placing shared memory between data agents and the database. Executable definitions learned in one session serve later agents and tasks, amortizing discovery, SQL construction, validation, and optimization across the workload. On TPC-DS, 16 agents use 72% fewer tokens and achieve a 4.6× end-to-end speedup relative to no memory. On six RelBench retraining tasks, token consumption falls by 69% on rel-stack and 50% on rel-trial from the second cutoff onward, with comparable predictive quality. Publication checks and shared maintenance additionally improve and preserve answer correctness: TPC-DS accuracy rises from 74% to 96% initially and reaches 99% after a restatement, compared with 55% without memory.

Future work. Building on star and snowflake metric definitions and entity-level aggregate features, future work will extend compilation to repeated-role joins and many-to-many bridges, and develop incremental feature maintenance. To address the unit/backup boundary in Section V-F, we will add explicit units and authoritative-source constraints, alongside checks on filter columns and value distributions, to support semantic reconciliation and drift detection.

Acknowledgment of AI-Generated Content

Evaluated agents and memory operations use DeepSeek-V4.1-Flash-1017 under the documented configurations. Under the authors' direction, Claude and ChatGPT assisted with coding, data analysis, visualization, and manuscript editing. The authors designed the study, verified all code, results, and text, and take full responsibility for the work.

References

[1] J. Robinson, R. Ranjan, W. Hu, K. Huang, J. Han, A. Dobles, M. Fey, J. E. Lenssen, Y. Yuan, Z. Zhang, X. He, and J. Leskovec, "RelBench: A Benchmark for Deep Learning on Relational Databases," in *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*, 2024, pp. 21 330–21 341. [Online]. Available: https://doi.org/10.52202/079017-0672

[2] BIRD Team, "LiveSQLBench: A Dynamic and Contamination-Free Benchmark for Evaluating LLMs on Real-World Text-to-SQL Tasks," 2025, dataset and documentation, releases base-lite (2025-05-30) and base-full-v1 (2025-09-04). Accessed 2026-10-14. [Online]. Available: https://github.com/bird-bench/livesqlbench

[3] N. Huo, X. Xu, J. Li, P. Jacobsson, S. Lin, B. Qin, B. Hui, X. Li, G. Qu, S. Si, L. Han, E. Alexander, X. Zhu, R. Qin, R. Yu, Y. Jin, F. Zhou, W. Zhong, Y. Chen, H. Liu, C. Ma, F. Ozcan, Y. Papakonstantinou, and R. Cheng, "BIRD-INTERACT: Re-imagining Text-to-SQL Evaluation via Lens of Dynamic Interactions," in *International Conference on Learning Representations (ICLR)*, 2026.

[4] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K. Chang, F. Huang, R. Cheng, and Y. Li, "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," in *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*, 2023, pp. 42 330–42 357.

[5] L. Ma, D. V. Aken, A. Hefny, G. Mezerhane, A. Pavlo, and G. J. Gordon, "Query-based Workload Forecasting for Self-Driving Database Management Systems," in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018, pp. 631–645. [Online]. Available: https://doi.org/10.1145/3183713.3196908

[6] A. van Renen, D. Horn, P. Pfeil, K. Vaidya, W. Dong, M. Narayanaswamy, Z. Liu, G. Saxena, A. Kipf, and T. Kraska, "Why TPC Is Not Enough: An Analysis of the Amazon Redshift Fleet," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 3694–3706, 2024. [Online]. Available: https://doi.org/10.14778/3681954.3682031

[7] Amazon Web Services, "Redset: Query Metadata of Amazon Redshift Clusters," 2024, dataset and documentation. Accessed 2026-10-01. [Online]. Available: https://github.com/amazon-science/redset

[8] S. Liu, S. Ponnapalli, S. Shankar, S. Zeighami, A. Zhu, S. Agarwal, R. Chen, S. Suwito, S. Yuan, I. Stoica, M. Zaharia, A. Cheung, N. Crooks, J. E. Gonzalez, and A. G. Parameswaran, "Supporting Our AI Overlords: Redesigning Data Systems to be Agent-First," in *16th Annual Conference on Innovative Data Systems Research (CIDR)*, 2026. [Online]. Available: https://www.cidrdb.org/cidr2026/papers/p32-liu.pdf

[9] A. Biswal, C. Lei, X. Qin, A. Li, B. Narayanaswamy, and T. Kraska, "AgentSM: Semantic Memory for Agentic Text-to-SQL," 2026, arXiv:2601.15709 (preprint). [Online]. Available: https://arxiv.org/abs/2601.15709

[10] G. Zhang, M. Fu, K. Wang, F. Wan, M. Yu, and S. Yan, "G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 12 988–13 018. [Online]. Available: https://doi.org/10.52202/085713-0439

[11] L. Weng, Y. Tang, Y. Feng, Z. Chang, R. Chen, H. Feng, C. Hou, D. Huang, Y. Li, H. Rao, H. Wang, C. Wei, X. Yang, Y. Zhang, Y. Zheng, X. Huang, M. Zhu, Y. Ma, B. Cui, P. Chen, and W. Chen, "DataLab: A Unified Platform for LLM-Powered Business Intelligence," in *Proceedings of the 41st IEEE International Conference on Data Engineering (ICDE)*, 2025, pp. 4346–4359. [Online]. Available: https://doi.org/10.1109/ICDE65448.2025.00326

[12] dbt Labs, "About MetricFlow," 2026, product documentation. Accessed 2026-10-23. [Online]. Available: https://docs.getdbt.com/docs/build/about-metricflow

[13] Databricks, "Model Metric Views (section "Model star schemas")," 2026, product documentation. Accessed 2026-10-23. [Online]. Available: https://docs.databricks.com/aws/en/uc-semantics/metric-views/basic-modeling#model-star-schemas

[14] S. Dar, M. J. Franklin, B. T. Jónsson, D. Srivastava, and M. Tan, "Semantic Data Caching and Replacement," in *Proceedings of the 22nd International Conference on Very Large Data Bases (VLDB)*, 1996, pp. 330–341.

[15] D. R. K. Ports, A. T. Clements, I. Zhang, S. Madden, and B. Liskov, "Transactional Consistency and Automatic Management in an Application Data Cache," in *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2010, pp. 279–292.

[16] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. John Wiley & Sons, 2013.

[17] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 3146–3154. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[18] Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li, U. Thakker, J. Zou, and K. Olukotun, "Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models," in *International Conference on Learning Representations (ICLR)*, 2026.

[19] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig, "Agent Workflow Memory," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. PMLR, Vol. 267, 2025, pp. 63 897–63 911.

[20] M. Fan, J. Fan, Y. Zhang, S. Zhang, X. Du, J. Song, P. Li, F. Jiang, T. Zhang, and J. Chen, "DeepPrep: An LLM-Powered Agentic System for Autonomous Data Preparation," *Proceedings of the VLDB Endowment*, vol. 19, no. 11, pp. 3371–3384, 2026. [Online]. Available: https://doi.org/10.14778/3836663.3836695

[21] J. Liang, C. Lei, X. Qin, J. Zhang, A. Katsifodimos, C. Faloutsos, and H. Rangwala, "FeatPilot: Automatic Feature Augmentation on Tabular Data," in *Proceedings of the 41st IEEE International Conference on Data Engineering (ICDE)*, 2025, pp. 2148–2160. [Online]. Available: https://doi.org/10.1109/ICDE65448.2025.00163

[22] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," in *ECAI 2025*, ser. Frontiers in Artificial Intelligence and Applications, Vol. 413, 2025, pp. 2993–3000. [Online]. Available: https://doi.org/10.3233/FAIA251160

[23] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," 2023, arXiv:2310.08560 (preprint). [Online]. Available: https://arxiv.org/abs/2310.08560

[24] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM Agents Are Experiential Learners," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024, pp. 19 632–19 642. [Online]. Available: https://doi.org/10.1609/aaai.v38i17.29936

[25] S. Ouyang, J. Yan, I.-H. Hsu, Y. Chen, K. Jiang, Z. Wang, R. Han, L. T. Le, S. Daruki, X. Tang, V. Tirumalashetty, G. Lee, M. Rofouei, H. Lin, J. Han, C.-Y. Lee, and T. Pfister, "ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory," in *International Conference on Learning Representations (ICLR)*, 2026.

[26] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-Mem: Agentic Memory for LLM Agents," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 20 004–20 031.

[27] Q. Zhang, M. Wornow, and K. Olukotun, "Agentic Plan Caching: Test-Time Memory for Fast and Cost-Efficient LLM Agents," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 114 372–114 398. [Online]. Available: https://doi.org/10.52202/085713-3451

[28] J. Fioresi, P. P. Kulkarni, A. Vayani, S. Wang, and M. Shah, "Learning to Share: Selective Memory for Efficient Parallel Agentic Systems," in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, ser. PMLR 306, 2026, pp. 31 146–31 160. [Online]. Available: https://proceedings.mlr.press/v306/fioresi26a.html

[29] A. Rezazadeh, Z. Li, A. Lou, Y. Zhao, W. Wei, and Y. Bao, "Collaborative Memory: Multi-User Memory Sharing in LLM Agents with Dynamic Access Control," 2025, arXiv:2505.18279 (preprint). [Online]. Available: https://arxiv.org/abs/2505.18279

[30] Y. Margalit, N. Cohen-Inger, E. Avram, R. Taig, and O. Margalit, "Governed Shared Memory for Multi-Agent LLM Systems," 2026, arXiv:2606.24535 (preprint). [Online]. Available: https://arxiv.org/abs/2606.24535

[31] M. Wu and A. Canedo, "Invalidation Contracts for Cross-Episode Agent Memory," 2026, arXiv:2609.00243 (preprint). [Online]. Available: https://arxiv.org/abs/2609.00243

[32] E. Chen, S. Wang, and C. G. Brinton, "Fresh Memory, Stale Plans: Derivation Currency for Distributed LLM-Agent Memory," 2026, arXiv:2609.03340 (preprint). [Online]. Available: https://arxiv.org/abs/2609.03340

[33] X. Li, Y. Wang, H. Lu, Z. Chen, M. Li, P. Song, M. Zheng, and T. Cai, "MemTX: Transactional Belief Commit for Stateful Agent Memory," 2026, arXiv:2607.23929 (preprint). [Online]. Available: https://arxiv.org/abs/2607.23929

[34] Y. Zhao, C. Dai, M. Kou, and Y. Xiu, "MEMOREPAIR: Barrier-First Cascade Repair in Agentic Memory," 2026, arXiv:2605.07242 (preprint). [Online]. Available: https://arxiv.org/abs/2605.07242

[35] Y. Luo, G. Li, J. Fan, and N. Tang, "Data Agents: Levels, State of the Art, and Open Problems," in *Companion of the 2026 International Conference on Management of Data (SIGMOD-Companion)*, 2026, pp. 571–579. [Online]. Available: https://doi.org/10.1145/3788853.3801878

[36] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin, H. Su, Z. Suo, H. Gao, W. Hu, P. Yin, V. Zhong, C. Xiong, R. Sun, Q. Liu, S. Wang, and T. Yu, "Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows," in *International Conference on Learning Representations (ICLR)*, 2025.

[37] A. Floratou, F. Psallidas, F. Zhao, S. Deep, G. Hagleither, W. Tan, J. Cahoon, R. Alotaibi, J. Henkel, A. Singla, A. van Grootel, B. Chow, K. Deng, K. Lin, M. Campos, V. Emani, V. Pandit, V. Shnayder, W. Wang, and C. Curino, "NL2SQL is a Solved Problem... Not!" in *14th Annual Conference on Innovative Data Systems Research (CIDR)*, 2024. [Online]. Available: https://www.cidrdb.org/cidr2024/papers/p74-floratou.pdf

[38] J. Sequeda, D. Allemang, and B. Jacob, "A Benchmark to Understand the Role of Knowledge Graphs on Large Language Model's Accuracy for Question Answering on Enterprise SQL Databases," in *Proceedings of the 7th Joint Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, 2024, pp. 1–12. [Online]. Available: https://doi.org/10.1145/3661304.3661901

[39] J. Jiang, H. Xie, S. Shen, Y. Shen, Z. Zhang, M. Lei, Y. Zheng, Y. Li, C. Li, D. Huang, Y. Wu, W. Zhang, B. Cui, and P. Chen, "SiriusBI: A Comprehensive LLM-Powered Solution for Data Analytics in Business Intelligence," *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 4860–4873, 2025. [Online]. Available: https://doi.org/10.14778/3750601.3750610

[40] M. Fey, W. Hu, K. Huang, J. E. Lenssen, R. Ranjan, J. Robinson, R. Ying, J. You, and J. Leskovec, "Position: Relational Deep Learning – Graph Representation Learning on Relational Databases," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. PMLR, Vol. 235, 2024, pp. 13 592–13 607. [Online]. Available: https://proceedings.mlr.press/v235/fey24a.html

[41] N. Hollmann, S. Müller, and F. Hutter, "Large Language Models for Automated Data Science: Introducing CAAFE for Context-Aware Automated Feature Engineering," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 44 753–44 775. [Online]. Available: https://doi.org/10.52202/075280-1938

[42] S. Guo, C. Deng, Y. Wen, H. Chen, Y. Chang, and J. Wang, "DS-Agent: Automated Data Science by Empowering Large Language Models with Case-Based Reasoning," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. PMLR, Vol. 235, 2024, pp. 16 813–16 848.

[43] H. Gao, S. Cai, T. T. A. Dinh, Z. Huang, and B. C. Ooi, "CtxPipe: Context-aware Data Preparation Pipeline Construction for Machine Learning," *Proceedings of the ACM on Management of Data*, vol. 2, no. 6, pp. 1–27, 2024. [Online]. Available: https://doi.org/10.1145/3698831

[44] Y. Ge, F. Li, Y. Mao, Y. Yang, C. Ge, Z. Chen, J. Long, and Y. Gao, "KnowTrans: Boosting Transferability of Data Preparation LLMs via Knowledge Augmentation," in *Proceedings of the 41st IEEE International Conference on Data Engineering (ICDE)*, 2025, pp. 2852–2866. [Online]. Available: https://doi.org/10.1109/ICDE65448.2025.00214

[45] T. K. Sellis, "Multiple-Query Optimization," *ACM Transactions on Database Systems*, vol. 13, no. 1, pp. 23–52, 1988. [Online]. Available: https://doi.org/10.1145/42201.42203

[46] S. Harizopoulos, V. Shkapenyuk, and A. Ailamaki, "QPipe: A Simultaneously Pipelined Relational Query Engine," in *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, 2005, pp. 383–394. [Online]. Available: https://doi.org/10.1145/1066157.1066201

[47] G. Giannikis, G. Alonso, and D. Kossmann, "SharedDB: Killing One Thousand Queries With One Stone," *Proceedings of the VLDB Endowment*, vol. 5, no. 6, pp. 526–537, 2012. [Online]. Available: https://doi.org/10.14778/2168651.2168654

[48] M. G. Ivanova, M. L. Kersten, N. J. Nes, and R. A. P. Gonçalves, "An Architecture for Recycling Intermediates in a Column-Store," *ACM Transactions on Database Systems*, vol. 35, no. 4, 2010, art. no. 24. [Online]. Available: https://doi.org/10.1145/1862919.1862921

[49] C. Garrod, A. Manjhi, A. Ailamaki, B. Maggs, T. Mowry, C. Olston, and A. Tomasic, "Scalable Query Result Caching for Web Applications," *Proceedings of the VLDB Endowment*, vol. 1, no. 1, pp. 550–561, 2008. [Online]. Available: https://doi.org/10.14778/1453856.1453917

[50] P. K. Gunda, L. Ravindranath, C. A. Thekkath, Y. Yu, and L. Zhuang, "Nectar: Automatic Management of Data and Computation in Datacenters," in *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2010, pp. 75–88.

[51] R. Nishtala, H. Fugal, S. Grimm, M. Kwiatkowski, H. Lee, H. C. Li, R. McElroy, M. Paleczny, D. Peek, P. Saab, D. Stafford, T. Tung, and V. Venkataramani, "Scaling Memcache at Facebook," in *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2013, pp. 385–398.

[52] A. Gupta and I. S. Mumick, "Maintenance of Materialized Views: Problems, Techniques, and Applications," *IEEE Data Engineering Bulletin*, vol. 18, no. 2, pp. 3–18, 1995.

[53] J. A. Blakeley, P.-Å. Larson, and F. W. Tompa, "Efficiently Updating Materialized Views," in *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data*, 1986, pp. 61–71. [Online]. Available: https://doi.org/10.1145/16894.16861

[54] L. S. Colby, T. Griffin, L. Libkin, I. S. Mumick, and H. Trickey, "Algorithms for Deferred View Maintenance," in *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*, 1996, pp. 469–480. [Online]. Available: https://doi.org/10.1145/233269.233364

[55] J. Zhou, P.-Å. Larson, and H. G. Elmongui, "Lazy Maintenance of Materialized Views," in *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB)*, 2007, pp. 231–242.

[56] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build Systems à la Carte," *Proceedings of the ACM on Programming Languages*, vol. 2, no. ICFP, 2018, art. no. 79. [Online]. Available: https://doi.org/10.1145/3236774

[57] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster, "Adapton: Composable, Demand-Driven Incremental Computation," in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014, pp. 156–166. [Online]. Available: https://doi.org/10.1145/2594291.2594324

[58] H.-J. Lenz and A. Shoshani, "Summarizability in OLAP and Statistical Data Bases," in *Proceedings of the 9th International Conference on Scientific and Statistical Database Management (SSDBM)*, 1997, pp. 132–143. [Online]. Available: https://doi.org/10.1109/SSDM.1997.621175

[59] J.-N. Mazón, J. Lechtenbörger, and J. Trujillo, "A Survey on Summarizability Issues in Multidimensional Modeling," *Data & Knowledge Engineering*, vol. 68, no. 12, pp. 1452–1469, 2009. [Online]. Available: https://doi.org/10.1016/j.datak.2009.07.010

[60] J. Horner, I.-Y. Song, and P. P. Chen, "An Analysis of Additivity in OLAP Systems," in *Proceedings of the 7th ACM International Workshop on Data Warehousing and OLAP (DOLAP)*, 2004, pp. 83–91. [Online]. Available: https://doi.org/10.1145/1031763.1031779

[61] J.-M. Nicolas, "Logic for Improving Integrity Checking in Relational Data Bases," *Acta Informatica*, vol. 18, no. 3, pp. 227–253, 1982. [Online]. Available: https://doi.org/10.1007/BF00263192

[62] S. Ceri and J. Widom, "Deriving Production Rules for Constraint Maintenance," in *Proceedings of the 16th International Conference on Very Large Data Bases (VLDB)*, 1990, pp. 566–577.

[63] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating Large-Scale Data Quality Verification," *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018. [Online]. Available: https://doi.org/10.14778/3229863.3229867

[64] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data Validation for Machine Learning," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 1, 2019, pp. 334–347.

[65] dbt Labs, "Add Data Tests to Your DAG," 2026, product documentation. Accessed 2026-10-02. [Online]. Available: https://docs.getdbt.com/docs/build/data-tests